

```
In [ ]: # Data Processing
```

```
In [88]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import os

# This is setting the directory of my code to be the github folder we created on our account

PANEL_URL = "https://raw.githubusercontent.com/truthisonegt-pixel/Data-ML-Repository-/main/Panel_Final_For_

df = pd.read_csv(PANEL_URL)
df["month"] = pd.to_datetime(df["month"])

# 1. CHECK DISTRIBUTIONS, this is checking the distributions of the variables (Dependent, Independent and other
variables_to_check = [
    'rev_index_route_month', # Dependent
    'freight_index', 'bunker', 'us_epu_index', 'capacity_index', # Main IVs
    'eu_ip_index', 'uk_ip_index', 'trade_imbalance', 'gbp_eur_rate', 'uk_cpi', 'teu_volume' # Controls
]

results = []

for var in variables_to_check:
    if var in panel.columns:
        data = panel[var].dropna()

        if len(data) == 0:
            print(f"Skipping {var} - no data")
            continue

        # Calculate statistics, We are checking the skewness of the data, looking if it is
        # skewed to any one of the tails of the distribution
```

```
# The kurtosis is also checking if we have any extreme outliers within our dataset

skewness = stats.skew(data)
kurtosis = stats.kurtosis(data)
cv = (data.std() / data.mean()) * 100 if data.mean() != 0 else 0

# Shapiro-Wilk test for normality, this is checking if our data is normally distributed,
# if not we can apply transformations to adjust the dataset to be more normal
if len(data) > 5000:
    sample = data.sample(5000, random_state=42)
else:
    sample = data

_, p_value = stats.shapiro(sample)

# Determine if transformation needed, after doing the statistical tests for our dataset,
#this is helping map which variables will actually
needs_transform = 'NO'
reason = 'Normal'

if abs(skewness) > 2:
    needs_transform = 'YES'
    reason = f'High skewness ({skewness:.2f})'
elif kurtosis > 7:
    needs_transform = 'MAYBE'
    reason = f'Heavy tails (kurtosis={kurtosis:.2f})'
elif cv > 100:
    needs_transform = 'YES'
    reason = f'High variability (CV={cv:.1f}%)'
elif p_value < 0.001:
    needs_transform = 'MAYBE'
    reason = f'Non-normal (p<0.001)'

results.append({
    'Variable': var,
    'Mean': f"{data.mean():.2f}",
    'Std': f"{data.std():.2f}",
    'Min': f"{data.min():.2f}",
```

```

        'Max': f"{data.max():.2f}",
        'Skewness': f"{skewness:.2f}",
        'Kurtosis': f"{kurtosis:.2f}",
        'Transform?': needs_transform,
        'Reason': reason
    })

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))

```

	Variable	Mean	Std	Min	Max	Skewness	Kurtosis	Transform?	
Reason									
rev_index_route_month	5187124.57	3008332.78	536501.28	26375661.09	1.33	3.85	MAYBE	Non-	
normal (p<0.001)									
freight_index	1215.33	722.39	222.88	5784.20	2.05	6.55	YES	High	
skewness (2.05)									
bunker	483.07	175.05	200.29	1051.24	1.04	1.24	MAYBE	Non-	
normal (p<0.001)									
us_epu_index	123.48	84.82	21.94	724.94	2.66	10.97	YES	High	
skewness (2.66)									
capacity_index	101.19	1.41	99.00	103.50	0.02	-1.20	MAYBE	Non-	
normal (p<0.001)									
eu_ip_index	0.06	2.65	-18.50	14.00	-1.71	24.01	MAYBE	Heavy tails	
(kurtosis=24.01)									
uk_ip_index	0.01	2.17	-15.60	10.70	-1.74	23.03	MAYBE	Heavy tails	
(kurtosis=23.03)									
trade_imbalance	-1003.66	1099.30	-4474.16	1173.43	-0.54	-0.13	MAYBE	Non-	
normal (p<0.001)									
gbp_eur_rate	1.21	0.13	1.05	1.71	2.07	3.41	YES	High	
skewness (2.07)									
uk_cpi	129.13	7.20	117.17	141.88	0.07	-1.20	MAYBE	Non-	
normal (p<0.001)									
teu_volume	4447.86	1408.41	1918.62	10141.71	0.78	0.19	MAYBE	Non-	
normal (p<0.001)									

```

In [83]: # 2. CHECK FOR ZEROS/NEGATIVES, we need a complete dataset, and also make sure that we do not
#
logarithmically transform any variables which have negative values

```

```

for var in variables_to_check:
    if var in panel.columns:
        data = panel[var]
        n_zeros = (data == 0).sum()
        n_neg = (data < 0).sum()

        if n_zeros > 0 or n_neg > 0:
            print(f"{var}:")
            if n_zeros > 0:
                print(f" • {n_zeros} zeros ({n_zeros/len(data)*100:.1f}%)")
            if n_neg > 0:
                print(f" • {n_neg} negative values ({n_neg/len(data)*100:.1f}%)")

            if n_neg > 0:
                print(f" → Cannot use LOG transformation")
            elif n_zeros > 0:
                print(f" → Use LOG(x + 1) or LOG(x + c) transformation")
            print()

```

eu_ip_index:

- 35 zeros (1.7%)
- 347 negative values (16.5%)
- Cannot use LOG transformation

uk_ip_index:

- 31 zeros (1.5%)
- 408 negative values (19.4%)
- Cannot use LOG transformation

trade_imbalance:

- 562 negative values (26.8%)
- Cannot use LOG transformation

In [87]: *# 3. CHECK SCALE DIFFERENCES, Our variables all have varying scales, hence we are making sure that all
our variables are of similar scale, if not we are transforming them to be more inline.*

```

print("SCALE COMPARISON (for interpretation)")

scale_check = []
for var in variables_to_check:
    if var in panel.columns:
        data = panel[var].dropna()
        if len(data) > 0 and data.mean() != 0:
            scale_check.append({
                'Variable': var,
                'Mean': data.mean(),
                'Order of Magnitude': int(np.log10(abs(data.mean())))
            })

if scale_check:
    scale_df = pd.DataFrame(scale_check).sort_values('Mean', ascending=False)
    print(scale_df.to_string(index=False))

```

```

SCALE COMPARISON (for interpretation)
      Variable      Mean  Order of Magnitude
rev_index_route_month  5.187125e+06           6
      teu_volume  4.447858e+03           3
    freight_index  1.215330e+03           3
        bunker  4.830715e+02           2
        uk_cpi  1.291348e+02           2
    us_epu_index  1.234768e+02           2
capacity_index  1.011923e+02           2
    gbp_eur_rate  1.209210e+00           0
    eu_ip_index  5.575758e-02          -1
    uk_ip_index  9.333333e-03          -2
trade_imbalance -1.003664e+03           3

```

In [86]: # 4. RECOMMENDED TRANSFORMATIONS, (this is just summarizing all of our vairables and the needed transformati

```

recommendations = {
    'rev_index_route_month': 'LOG (dependent variable - highly skewed revenue data)',
    'freight_index': 'LOG (price indices are typically log-transformed)',
    'bunker': 'LOG (commodity prices are typically log-transformed)',

```

```

'teu_volume': 'LOG (trade volumes are typically log-transformed)',
'us_epu_index': 'LOG or KEEP (policy uncertainty – check skewness)',
'trade_imbalance': 'KEEP (can be negative, measures direction)',
'eu_ip_index': 'KEEP (already an index)',
'uk_ip_index': 'KEEP (already an index)',
'gbp_eur_rate': 'LOG (exchange rates often log-transformed)',
'uk_cpi': 'LOG (price level – typically log-transformed)',
'capacity_index': 'KEEP (already an index)',
'congestion_dummy': 'KEEP (binary variable)'
}
for var, rec in recommendations.items():
    if var in panel.columns:
        print(f"• {var}: {rec}")

```

- rev_index_route_month: LOG (dependent variable – highly skewed revenue data)
- freight_index: LOG (price indices are typically log-transformed)
- bunker: LOG (commodity prices are typically log-transformed)
- teu_volume: LOG (trade volumes are typically log-transformed)
- us_epu_index: LOG or KEEP (policy uncertainty – check skewness)
- trade_imbalance: KEEP (can be negative, measures direction)
- eu_ip_index: KEEP (already an index)
- uk_ip_index: KEEP (already an index)
- gbp_eur_rate: LOG (exchange rates often log-transformed)
- uk_cpi: LOG (price level – typically log-transformed)
- capacity_index: KEEP (already an index)
- congestion_dummy: KEEP (binary variable)

In [85]: # 5. CREATE TRANSFORMED VARIABLES

```
print("CREATING LOG-TRANSFORMED VARIABLES")
```

```

# Log transform appropriate variables, this is transforming the variables with a natural logarithmic transform
log_vars = ['rev_index_route_month', 'freight_index', 'bunker', 'teu_volume',
            'us_epu_index', 'gbp_eur_rate', 'uk_cpi']

```

```
for var in log_vars:
```

```

if var in panel.columns:
    # Check for zeros/negatives
    min_val = panel[var].min()

    if pd.isna(min_val):
        print(f"× Skipped {var} (all NaN)")
        continue

    if min_val > 0:
        panel[f'ln_{var}'] = np.log(panel[var])
        print(f"✓ Created ln_{var} (natural log)")
    elif min_val == 0:
        panel[f'ln_{var}'] = np.log(panel[var] + 1)
        print(f"✓ Created ln_{var} using log(x+1) transformation")
    else:
        print(f"× Skipped {var} (contains negative values: min={min_val:.2f})")

```

CREATING LOG-TRANSFORMED VARIABLES

- ✓ Created ln_rev_index_route_month (natural log)
- ✓ Created ln_freight_index (natural log)
- ✓ Created ln_bunker (natural log)
- ✓ Created ln_teu_volume (natural log)
- ✓ Created ln_us_epu_index (natural log)
- ✓ Created ln_gbp_eur_rate (natural log)
- ✓ Created ln_uk_cpi (natural log)

In [84]: *# 6. CORRELATION CHECK, to ensure that our variables are not too correlated with one another, we are running*

```

print("MULTICOLLINEARITY CHECK")

# Select numeric columns for correlation
numeric_cols = [var for var in variables_to_check if var in panel.columns]
corr_matrix = panel[numeric_cols].corr()

# Check for high correlations (>0.8)
high_corr = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):

```

```

        if abs(corr_matrix.iloc[i, j]) > 0.8:
            high_corr.append({
                'Var1': corr_matrix.columns[i],
                'Var2': corr_matrix.columns[j],
                'Correlation': f"{corr_matrix.iloc[i, j]:.3f}"
            })

    if high_corr:
        print("HIGH CORRELATIONS DETECTED (may cause multicollinearity):\n")
        high_corr_df = pd.DataFrame(high_corr)
        print(high_corr_df.to_string(index=False))
    else:
        print("✓ No severe multicollinearity detected (all |r| < 0.8)")

    # SAVE TRANSFORMED PANEL (This is just saving our model as a xlsx file and CSV)
    panel.to_csv('Shipping_Volume_Panel_Transformed.csv', index=False)
    panel.to_excel('Shipping_Volume_Panel_Transformed.xlsx', index=False)

    new_cols = [col for col in panel.columns if col.startswith('ln_')]

```

MULTICOLLINEARITY CHECK

HIGH CORRELATIONS DETECTED (may cause multicollinearity):

	Var1	Var2	Correlation
rev_index_route_month	freight_index		0.804
capacity_index	uk_cpi		0.995

In [2]: pip install linearmodels


```

Requirement already satisfied: linearmodels in /opt/anaconda3/lib/python3.13/site-packages (7.0)
Requirement already satisfied: numpy<3,>=1.22.3 in /opt/anaconda3/lib/python3.13/site-packages (from linear
models) (2.3.5)
Requirement already satisfied: pandas>=1.4.0 in /opt/anaconda3/lib/python3.13/site-packages (from linearmod
els) (2.3.3)
Requirement already satisfied: scipy>=1.8.0 in /opt/anaconda3/lib/python3.13/site-packages (from linearmode
ls) (1.16.3)
Requirement already satisfied: statsmodels>=0.13.0 in /opt/anaconda3/lib/python3.13/site-packages (from lin
earmodels) (0.14.5)
Requirement already satisfied: mpy_extensions>=0.4 in /opt/anaconda3/lib/python3.13/site-packages (from li
nearmodels) (1.0.0)
Requirement already satisfied: pyhdfe>=0.1 in /opt/anaconda3/lib/python3.13/site-packages (from linearmodel
s) (0.2.0)
Requirement already satisfied: formulaic>=1.2.1 in /opt/anaconda3/lib/python3.13/site-packages (from linear
models) (1.2.1)
Requirement already satisfied: interface-meta>=1.2.0 in /opt/anaconda3/lib/python3.13/site-packages (from f
ormulaic>=1.2.1->linearmodels) (1.3.0)
Requirement already satisfied: narwhals>=1.17 in /opt/anaconda3/lib/python3.13/site-packages (from formulai
c>=1.2.1->linearmodels) (2.7.0)
Requirement already satisfied: typing-extensions>=4.2.0 in /opt/anaconda3/lib/python3.13/site-packages (fro
m formulaic>=1.2.1->linearmodels) (4.15.0)
Requirement already satisfied: wrapt>=1.17.0rc1 in /opt/anaconda3/lib/python3.13/site-packages (from formul
aic>=1.2.1->linearmodels) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/anaconda3/lib/python3.13/site-packages (from
pandas>=1.4.0->linearmodels) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/anaconda3/lib/python3.13/site-packages (from pandas>=1.
4.0->linearmodels) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/anaconda3/lib/python3.13/site-packages (from pandas>=
1.4.0->linearmodels) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/site-packages (from python-dateuti
l>=2.8.2->pandas>=1.4.0->linearmodels) (1.17.0)
Requirement already satisfied: patsy>=0.5.6 in /opt/anaconda3/lib/python3.13/site-packages (from statsmodel
s>=0.13.0->linearmodels) (1.0.1)
Requirement already satisfied: packaging>=21.3 in /opt/anaconda3/lib/python3.13/site-packages (from statsmo
dels>=0.13.0->linearmodels) (25.0)
Note: you may need to restart the kernel to use updated packages.

```

```
In [ ]: # First Demand Model (OLS Panel Regression Model)
```

```

In [89]: from statsmodels.regression.linear_model import OLS
from statsmodels.tools.tools import add_constant
from statsmodels.stats.diagnostic import acorr_breusch_godfrey
from statsmodels.stats.stattools import durbin_watson
import warnings
warnings.filterwarnings("ignore")

# 0. Loading the data from GitHub, we are loading it from our online shared folder for our data.

PANEL_URL = "https://raw.githubusercontent.com/truthisonegt-pixel/Data-ML-Repository-/main/Shipping_Volume_"

df = pd.read_csv(PANEL_URL)
df["month"] = pd.to_datetime(df["month"])

# 1. VERIFY REQUIRED VARIABLES EXIST

# This is a sanity check, just making sure that all of our variables are present before running our demand
# If our log-transformed variables are not present, it will attempt to make them.

required_vars = [
    'ln_teu_volume',      # Dependent variable: log of TEU volume (shipping container volumes per route)
    'ln_freight_index',   # IV: log of Baltic freight rate index
    'ln_bunker',          # IV: log of bunker crude fuel price
    'ln_us_epu_index',     # IV: log of US Economic Policy Uncertainty
    'congestion_dummy',   # IV: binary indicator for port congestion periods at these ports
    'eu_ip_index',        # Control: EU industrial production index
    'uk_ip_index',        # Control: UK industrial production index
    'trade_imbalance',    # Control: UK-EU trade balance
    'ln_gbp_eur_rate',    # Control: log of GBP/EUR exchange rate
    'ln_uk_cpi',          # Control: log of UK CPI
    'route',              # Route identifier (UK-France, UK-Germany, UK-Netherlands)
    'direction'           # Direction of Trade: Exports or Imports
]

missing_vars = [var for var in required_vars if var not in df.columns]

```

```

if missing_vars:
    print(f"Missing variables: {missing_vars}")
    print("\nAttempting to create missing variables...")

    # Create ln_teu_volume if missing (This replaces the zeros with a NaN)
    if 'ln_teu_volume' not in df.columns and 'teu_volume' in df.columns:
        df['ln_teu_volume'] = np.log(df['teu_volume'].replace(0, np.nan))
        print("Created ln_teu_volume")

    # Map UK EPU index to US EPU index if missing
    if 'ln_us_epu_index' not in df.columns and 'ln_uk_epu_index' in df.columns:
        df['ln_us_epu_index'] = df['ln_uk_epu_index']
        print("Mapped ln_uk_epu_index to ln_us_epu_index")
else:
    print("All required variables present\n")

```

All required variables present

In [90]: # 2. Variables

```

# DEPENDENT VARIABLE
y_var = "ln_teu_volume" # Log of TEU volume (Twenty-foot Equivalent Units)

# MAIN INDEPENDENT VARIABLES (already log-transformed, except for our congestion_dummy)
main_iv_list = [
    "ln_freight_index",
    "ln_bunker",
    "ln_us_epu_index",
    "congestion_dummy"
]

# CONTROL VARIABLES (macro and FX factors)
control_vars = [
    "eu_ip_index",
    "uk_ip_index",
    "trade_imbalance",

```

```

    "ln_gbp_eur_rate",
    "ln_uk_cpi"
]

# This is just verifying that the direction column exists (Exports vs Imports)
if 'direction' not in df.columns:
    print("'direction' column not found. Creating bidirectional analysis...")
    df['direction'] = 'Both'

routes = df["route"].unique()
directions = df["direction"].unique()

# Storage for results from the model just ran
individual_results = []
full_model_results = []

print("VOLUME REGRESSION MODELS: DEMAND ANALYSIS")

print(f"\nDependent Variable: {y_var}")
print("Interpretation: Coefficients represent elasticities or percentage effects")

print()

```

VOLUME REGRESSION MODELS: DEMAND ANALYSIS

Dependent Variable: ln_teu_volume

Interpretation: Coefficients represent elasticities or percentage effects

In [91]: # 3. ROUTE-SPECIFIC REGRESSIONS

```

# For each route-direction combination (e.g., UK-Germany Exports):
# - This is initially subsetting the data
# - Then it is cleaning the data, converting it to a numeric value and dropping any non-existing data cells
# - Then we estimate the individual IV models (one main IV and the controls)
# - Then we do it for the complete model (all main IVs + controls)
# - Finally we do some diagnostics to ensure that the results of our analysis are statistically relevant

```

```

# (Durbin-Watson, Breusch-Godfrey for autocorrelation)
# - We choose to do simple linear models to prevent overcomplicating the models and allowing for simpler in

for route in routes:
    for direction in directions:
        print("="*80)
        print(f"ROUTE: {route} - {direction}")
        print("="*80)

        # Subset data for the relevant route-direction
        sub = df[(df["route"] == route) & (df["direction"] == direction)].copy()
        if len(sub) < 30:
            print(f"Insufficient data: {len(sub)} observations (need ≥30). Skipping...\n")
            continue

        # This is converting all the variables to numeric and dropping rows with missing values
        all_vars = [y_var] + main_iv_list + control_vars
        sub_clean = sub[all_vars].apply(pd.to_numeric, errors="coerce").dropna()

        if len(sub_clean) < 20:
            print(f"After cleaning: {len(sub_clean)} observations (need ≥20). Skipping...\n")
            continue

        print(f"\nSample size: {len(sub_clean)} observations")
        print(f"Period: {sub['month'].min().strftime('%Y-%m')} to {sub['month'].max().strftime('%Y-%m')}")

        # Descriptive statistics for dependent variable
        print(f"\nTEU Volume Statistics:")
        print(f"  Mean: {np.exp(sub_clean[y_var].mean()):,.0f} TEUs")
        print(f"  Std Dev: {sub_clean[y_var].std():.3f} (log)")
        print(f"  Min: {np.exp(sub_clean[y_var].min()):,.0f} TEUs")
        print(f"  Max: {np.exp(sub_clean[y_var].max()):,.0f} TEUs")

        # MODEL A: INDIVIDUAL IV MODELS (one main IV + controls at a time)

        # Purpose: This is essentially just isolating the effect of each main explanatory variable while er
        # to control for macroeconomic factors. We want to see which variables are driving shipping volumes

```

```

print("INDIVIDUAL IV MODELS (demand elasticities with Controls)")

for main_iv in main_iv_list:

    X_vars = [main_iv] + control_vars

    y = sub_clean[y_var]
    X = sub_clean[X_vars]
    X_const = add_constant(X) # Add intercept

    # Estimate OLS with HAC (Newey-West) standard errors, this is to help adjust for possible
    # heteroskedascity and autocorrelationn of the residuals in our model
    model = OLS(y, X_const).fit(cov_type="HAC", cov_kws={"maxlags": 4})

    # Extracting the main coefficient, p-value, and standard errors for the main IV
    coef = model.params[main_iv]
    pval = model.pvalues[main_iv]
    se = model.bse[main_iv]
    stars = "***" if pval < 0.01 else "*" if pval < 0.05 else "" if pval < 0.10 else ""

    # Helps interpret the elastacities
    if "ln_" in main_iv:

        interp = f"1% ↑ in {main_iv.replace('ln_', '')} → {coef:.2f}% change in volume"
    elif main_iv == "congestion_dummy":

        interp = f"Congestion changes volume by {(np.exp(coef)-1)*100:.1f}%"
    else:

        interp = f"1 unit ↑ in {main_iv} → {coef:.4f} change in ln(volume)"

    print(f"\n{'='*60}")
    print(f"Model: {main_iv} + controls")
    print(f"{'='*60}")
    print(f"  Variables: {main_iv}, {'', '}.join(control_vars)}")

```

```

print(f" Coefficient: {coef:.4f}{stars}")
print(f" Std Error (HAC): {se:.4f}")
print(f" P-value: {pval:.4f}")
print(f" R2: {model.rsquared:.4f}")
print(f" Interpretation: {interp}")

# Diagnostic tests for autocorrelation
# Durbin-Watson: 2 indicates no autocorrelation: <1 or >3 indicates that there is possible autocorrelation
dw = durbin_watson(model.resid)
# Breusch-Godfrey: tests for higher-order autocorrelation: a low p-value indicates autocorrelation
bg_test = acorr_breusch_godfrey(model, nlags=4)
print(f" Durbin-Watson: {dw:.3f}")
print(f" Breusch-Godfrey p: {bg_test[1]:.4f}")

# Store results for export
individual_results.append({
    "route": route,
    "direction": direction,
    "main_iv": main_iv,
    "coef": coef,
    "se_hac": se,
    "pval": pval,
    "r2": model.rsquared,
    "n_obs": len(sub_clean),
    "dw_stat": dw,
    "interpretation": interp
})

# MODEL B: FULL MODEL (all main IVs + controls simultaneously)

# Purpose: estimate the conjoint effect of all main explanatory variables together on the shipping cost

print("\n" + "="*80)
print("FULL DEMAND MODEL (all factors simultaneously)")
print("="*80)

# Regression: Y = all main IVs + controls

```

```

X_full = sub_clean[main_iv_list + control_vars]
y = sub_clean[y_var]
X_full_const = add_constant(X_full)

# This is estimating the full OLS model with standard errors (HAC)
full_model = OLS(y, X_full_const).fit(cov_type="HAC", cov_kwds={"maxlags": 4})

print(f"\nVariables: {'', '.join(main_iv_list + control_vars)}")
print(f"R²: {full_model.rsquared:.4f}")
print(f"Adjusted R²: {full_model.rsquared_adj:.4f}")
print(f"N: {len(sub_clean)}")
print(f"\nCoefficients (with HAC SE):")
print(f"{'Variable':<25} {'Coef':>10} {'Stars':<5} {'SE':>10} {'P-value':>10}")
print("-"*65)

# This is printing the coefficients for main IVs only from the model
for var in main_iv_list:
    coef_full = full_model.params[var]
    pval_full = full_model.pvalues[var]
    se_full = full_model.bse[var]
    stars_full = "***" if pval_full < 0.01 else "*" if pval_full < 0.05 else "" if pval_full < 0.1 else ""

    print(f"{'var':<25} {'coef_full':>10.4f} {'stars_full':<5} {'se_full':>10.4f} {'pval_full':>10.4f}")

# Interpretation
if "ln_" in var:
    interp_full = f"1% ↑ → {coef_full:.2f}% Δ volume"
elif var == "congestion_dummy":
    interp_full = f"Event: {(np.exp(coef_full)-1)*100:.1f}% volume change"
else:
    interp_full = f"1 unit ↑ → {coef_full:.4f} Δ ln(vol)"

print(f" → {interp_full}")

# Store full model results to export
full_model_results.append({
    "route": route,
    "direction": direction,

```



```

        "variable": var,
        "coef": coef_full,
        "se_hac": se_full,
        "pval": pval_full,
        "r2": full_model.rsquared,
        "r2_adj": full_model.rsquared_adj,
        "n_obs": len(sub_clean)
    })

    print()

```

```

=====
ROUTE: UK-DE – Exports
=====

```

Sample size: 117 observations
 Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 4,663 TEUs
 Std Dev: 0.162 (log)
 Min: 3,120 TEUs
 Max: 6,845 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

```

=====
Model: ln_freight_index + controls
=====

```

Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: -0.0520
 Std Error (HAC): 0.0436
 P-value: 0.2334
 R²: 0.2333
 Interpretation: 1% ↑ in freight_index → -0.05% change in volume
 Durbin-Watson: 1.759
 Breusch-Godfrey p: 0.0009

```

=====

```

Model: ln_bunker + controls

```
=====
Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: 0.1456**
Std Error (HAC): 0.0612
P-value: 0.0174
R²: 0.2851
Interpretation: 1% ↑ in bunker → 0.15% change in volume
Durbin-Watson: 1.944
Breusch-Godfrey p: 0.0449
=====
```

Model: ln_us_epu_index + controls

```
=====
Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.0714*
Std Error (HAC): 0.0367
P-value: 0.0515
R²: 0.2450
Interpretation: 1% ↑ in us_epu_index → -0.07% change in volume
Durbin-Watson: 1.863
Breusch-Godfrey p: 0.0073
=====
```

Model: congestion_dummy + controls

```
=====
Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.0834*
Std Error (HAC): 0.0487
P-value: 0.0869
R²: 0.2549
Interpretation: Congestion changes volume by -8.0%
Durbin-Watson: 1.862
Breusch-Godfrey p: 0.0079
=====
```

FULL DEMAND MODEL (all factors simultaneously)

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 R²: 0.3374
 Adjusted R²: 0.2817
 N: 117

Coefficients (with HAC SE):

Variable	Coef Stars	SE	P-value
ln_freight_index	-0.0941 ***	0.0268	0.0004
→ 1% ↑ → -0.09% Δ volume			
ln_bunker	0.1647 **	0.0751	0.0284
→ 1% ↑ → 0.16% Δ volume			
ln_us_epu_index	-0.0385	0.0376	0.3047
→ 1% ↑ → -0.04% Δ volume			
congestion_dummy	0.0074	0.0430	0.8631
→ Event: 0.7% volume change			

=====

ROUTE: UK-DE – Imports

=====

Sample size: 117 observations
 Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 6,535 TEUs
 Std Dev: 0.169 (log)
 Min: 3,654 TEUs
 Max: 10,142 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

=====

Model: ln_freight_index + controls

=====

Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: -0.0326

Std Error (HAC): 0.0349

P-value: 0.3498

R²: 0.5847

Interpretation: 1% ↑ in freight_index → -0.03% change in volume

Durbin-Watson: 1.755

Breusch-Godfrey p: 0.0001

=====
Model: ln_bunker + controls
=====

Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: 0.1287***

Std Error (HAC): 0.0489

P-value: 0.0085

R²: 0.6279

Interpretation: 1% ↑ in bunker → 0.13% change in volume

Durbin-Watson: 2.012

Breusch-Godfrey p: 0.0135

=====
Model: ln_us_epu_index + controls
=====

Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: -0.0600**

Std Error (HAC): 0.0302

P-value: 0.0467

R²: 0.5972

Interpretation: 1% ↑ in us_epu_index → -0.06% change in volume

Durbin-Watson: 1.903

Breusch-Godfrey p: 0.0012

=====
Model: congestion_dummy + controls
=====

Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: -0.0741*

Std Error (HAC): 0.0379

P-value: 0.0508

R^2 : 0.6066

Interpretation: Congestion changes volume by -7.1%

Durbin-Watson: 1.909

Breusch-Godfrey p: 0.0019

```
=====
FULL DEMAND MODEL (all factors simultaneously)
=====
```

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

R^2 : 0.6527

Adjusted R^2 : 0.6235

N: 117

Coefficients (with HAC SE):

Variable	Coef Stars	SE	P-value
ln_freight_index	-0.0676 ***	0.0196	0.0006
→ 1% ↑ → -0.07% Δ volume			
ln_bunker	0.1417 **	0.0584	0.0152
→ 1% ↑ → 0.14% Δ volume			
ln_us_epu_index	-0.0272	0.0271	0.3150
→ 1% ↑ → -0.03% Δ volume			
congestion_dummy	0.0043	0.0306	0.8879
→ Event: 0.4% volume change			

```
=====
ROUTE: UK-DE - nan
=====
```

Insufficient data: 0 observations (need ≥30). Skipping...

```
=====
ROUTE: UK-FR - Exports
=====
```

Sample size: 117 observations

Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 3,276 TEUs

Std Dev: 0.202 (log)

Min: 1,991 TEUs

Max: 5,177 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

=====

Model: ln_freight_index + controls

=====

Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: -0.0437

Std Error (HAC): 0.0650

P-value: 0.5020

R²: 0.3152

Interpretation: 1% ↑ in freight_index → -0.04% change in volume

Durbin-Watson: 1.272

Breusch-Godfrey p: 0.0000

=====

Model: ln_bunker + controls

=====

Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: 0.2949***

Std Error (HAC): 0.0654

P-value: 0.0000

R²: 0.4875

Interpretation: 1% ↑ in bunker → 0.29% change in volume

Durbin-Watson: 1.675

Breusch-Godfrey p: 0.0001

=====

Model: ln_us_epu_index + controls

=====

Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: -0.0885

Std Error (HAC): 0.0599

P-value: 0.1395

R²: 0.3353

Interpretation: 1% ↑ in us_epu_index → -0.09% change in volume

Durbin-Watson: 1.389

Breusch-Godfrey p: 0.0000

=====

Model: congestion_dummy + controls

=====

Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

Coefficient: -0.1597**

Std Error (HAC): 0.0668

P-value: 0.0167

R²: 0.3965

Interpretation: Congestion changes volume by -14.8%

Durbin-Watson: 1.480

Breusch-Godfrey p: 0.0000

=====

FULL DEMAND MODEL (all factors simultaneously)

=====

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

R²: 0.5388

Adjusted R²: 0.5000

N: 117

Coefficients (with HAC SE):

Variable	Coef Stars	SE	P-value
ln_freight_index	-0.1158 ***	0.0376	0.0020
→ 1% ↑ → -0.12% Δ volume			
ln_bunker	0.3821 ***	0.0785	0.0000
→ 1% ↑ → 0.38% Δ volume			
ln_us_epu_index	0.0090	0.0509	0.8597
→ 1% ↑ → 0.01% Δ volume			
congestion_dummy	0.0351	0.0506	0.4887

→ Event: 3.6% volume change

=====

ROUTE: UK-FR – Imports

=====

Sample size: 117 observations
Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 3,619 TEUs
Std Dev: 0.259 (log)
Min: 1,919 TEUs
Max: 7,020 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

=====

Model: ln_freight_index + controls

=====

Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.0444
Std Error (HAC): 0.0652
P-value: 0.4960
R²: 0.6378
Interpretation: 1% ↑ in freight_index → -0.04% change in volume
Durbin-Watson: 1.100
Breusch-Godfrey p: 0.0000

=====

Model: ln_bunker + controls

=====

Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: 0.2885***
Std Error (HAC): 0.0629
P-value: 0.0000
R²: 0.7379
Interpretation: 1% ↑ in bunker → 0.29% change in volume
Durbin-Watson: 1.504

Breusch–Godfrey p: 0.0000

=====

Model: ln_us_epu_index + controls

=====

Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: -0.0882
 Std Error (HAC): 0.0618
 P-value: 0.1538
 R²: 0.6497
 Interpretation: 1% ↑ in us_epu_index → -0.09% change in volume
 Durbin–Watson: 1.213
 Breusch–Godfrey p: 0.0000

=====

Model: congestion_dummy + controls

=====

Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: -0.1555**
 Std Error (HAC): 0.0635
 P-value: 0.0144
 R²: 0.6844
 Interpretation: Congestion changes volume by -14.4%
 Durbin–Watson: 1.292
 Breusch–Godfrey p: 0.0000

=====

FULL DEMAND MODEL (all factors simultaneously)

=====

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 R²: 0.7688
 Adjusted R²: 0.7494
 N: 117

Coefficients (with HAC SE):

Variable	Coef	Stars	SE	P-value
----------	------	-------	----	---------

```

-----
ln_freight_index      -0.1156 ***      0.0363      0.0015
  → 1% ↑ → -0.12% Δ volume
ln_bunker             0.3745 ***      0.0725      0.0000
  → 1% ↑ → 0.37% Δ volume
ln_us_epu_index       0.0063          0.0526      0.9041
  → 1% ↑ → 0.01% Δ volume
congestion_dummy      0.0358          0.0454      0.4306
  → Event: 3.6% volume change

```

```

=====
ROUTE: UK-FR – nan
=====

```

```

Insufficient data: 0 observations (need ≥30). Skipping...

```

```

=====
ROUTE: UK-NL – Exports
=====

```

Sample size: 117 observations
 Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 3,680 TEUs
 Std Dev: 0.223 (log)
 Min: 2,258 TEUs
 Max: 6,000 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

```

=====
Model: ln_freight_index + controls
=====

```

Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: 0.0121
 Std Error (HAC): 0.0517
 P-value: 0.8143
 R²: 0.5106
 Interpretation: 1% ↑ in freight_index → 0.01% change in volume

Durbin-Watson: 1.379
Breusch-Godfrey p: 0.0000

=====
Model: ln_bunker + controls
=====

Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: 0.3555***
Std Error (HAC): 0.0309
P-value: 0.0000
R²: 0.7250
Interpretation: 1% ↑ in bunker → 0.36% change in volume
Durbin-Watson: 2.316
Breusch-Godfrey p: 0.0167

=====
Model: ln_us_epu_index + controls
=====

Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.1108***
Std Error (HAC): 0.0427
P-value: 0.0095
R²: 0.5464
Interpretation: 1% ↑ in us_epu_index → -0.11% change in volume
Durbin-Watson: 1.486
Breusch-Godfrey p: 0.0000

=====
Model: congestion_dummy + controls
=====

Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.2317***
Std Error (HAC): 0.0432
P-value: 0.0000
R²: 0.6657
Interpretation: Congestion changes volume by -20.7%
Durbin-Watson: 1.926
Breusch-Godfrey p: 0.0005

```
=====
FULL DEMAND MODEL (all factors simultaneously)
=====
```

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi

R²: 0.7462

Adjusted R²: 0.7249

N: 117

Coefficients (with HAC SE):

Variable	Coef Stars	SE	P-value
ln_freight_index	-0.0713 ***	0.0260	0.0061
→ 1% ↑ → -0.07% Δ volume			
ln_bunker	0.3392 ***	0.0434	0.0000
→ 1% ↑ → 0.34% Δ volume			
ln_us_epu_index	0.0109	0.0269	0.6866
→ 1% ↑ → 0.01% Δ volume			
congestion_dummy	-0.0557 *	0.0321	0.0821
→ Event: -5.4% volume change			

```
=====
ROUTE: UK-NL – Imports
=====
```

Sample size: 117 observations

Period: 2016-01 to 2025-09

TEU Volume Statistics:

Mean: 4,367 TEUs

Std Dev: 0.229 (log)

Min: 2,618 TEUs

Max: 7,013 TEUs

INDIVIDUAL IV MODELS (demand elasticities with Controls)

```
=====
```

Model: ln_freight_index + controls

```
=====
Variables: ln_freight_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.0022
Std Error (HAC): 0.0495
P-value: 0.9646
R²: 0.6399
Interpretation: 1% ↑ in freight_index → -0.00% change in volume
Durbin-Watson: 1.314
Breusch-Godfrey p: 0.0000
```

Model: ln_bunker + controls

```
=====
Variables: ln_bunker, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: 0.3057***
Std Error (HAC): 0.0275
P-value: 0.0000
R²: 0.7913
Interpretation: 1% ↑ in bunker → 0.31% change in volume
Durbin-Watson: 2.163
Breusch-Godfrey p: 0.0070
```

Model: ln_us_epu_index + controls

```
=====
Variables: ln_us_epu_index, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
Coefficient: -0.0862**
Std Error (HAC): 0.0401
P-value: 0.0315
R²: 0.6608
Interpretation: 1% ↑ in us_epu_index → -0.09% change in volume
Durbin-Watson: 1.405
Breusch-Godfrey p: 0.0000
```

Model: congestion_dummy + controls

Variables: congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 Coefficient: -0.2007***
 Std Error (HAC): 0.0397
 P-value: 0.0000
 R²: 0.7512
 Interpretation: Congestion changes volume by -18.2%
 Durbin-Watson: 1.802
 Breusch-Godfrey p: 0.0000

=====

FULL DEMAND MODEL (all factors simultaneously)

=====

Variables: ln_freight_index, ln_bunker, ln_us_epu_index, congestion_dummy, eu_ip_index, uk_ip_index, trade_imbalance, ln_gbp_eur_rate, ln_uk_cpi
 R²: 0.8127
 Adjusted R²: 0.7969
 N: 117

Coefficients (with HAC SE):

Variable	Coef Stars	SE	P-value
ln_freight_index	-0.0739 ***	0.0255	0.0037
→ 1% ↑ → -0.07% Δ volume			
ln_bunker	0.3026 ***	0.0377	0.0000
→ 1% ↑ → 0.30% Δ volume			
ln_us_epu_index	0.0187	0.0286	0.5130
→ 1% ↑ → 0.02% Δ volume			
congestion_dummy	-0.0485	0.0326	0.1367
→ Event: -4.7% volume change			

=====

ROUTE: UK-NL – nan

=====

Insufficient data: 0 observations (need ≥30). Skipping...

=====

ROUTE: nan – Exports

```
=====
Insufficient data: 0 observations (need ≥30). Skipping...
```

```
=====
ROUTE: nan – Imports
=====
```

```
Insufficient data: 0 observations (need ≥30). Skipping...
```

```
=====
ROUTE: nan – nan
=====
```

```
Insufficient data: 0 observations (need ≥30). Skipping...
```

```
In [92]: # 4. SAVE RESULTS TO CSV
```

```
# We export all the estimated coefficients to CSV files
```

```
if len(individual_results) > 0:
    individual_df = pd.DataFrame(individual_results)
    individual_df.to_csv("Volume_Individual_IV_Models.csv", index=False)
    print("Saved: Volume_Individual_IV_Models.csv")
else:
    print("No individual results to save")

if len(full_model_results) > 0:
    full_df = pd.DataFrame(full_model_results)
    full_df.to_csv("Volume_Full_Model_Results.csv", index=False)
    print("Saved: Volume_Full_Model_Results.csv")
else:
    print("No full model results to save")
```

```
Saved: Volume_Individual_IV_Models.csv
```

```
Saved: Volume_Full_Model_Results.csv
```

```
In [ ]: # Model 1 Predictive Model for 2023–2024
```

```
In [96]: # This code is fitting a speerate OLS linear regression for each route & direction subset using the data up
```

```

# it then evaluates forecasts between 2023-2024
# It saves the per route coefficient and test-set predictions.
# This model is just being done for overall validation.
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# 0. LOAD DATA FROM our online GITHUB folder

PANEL_URL = "https://raw.githubusercontent.com/truthisonet-pixel/Data-ML-Repository-/main/Shipping_Volume_

df = pd.read_csv(PANEL_URL)
df["month"] = pd.to_datetime(df["month"])

# Keep only columns needed for regression and drop any rows with missing values
cols_needed = [
    "month", "route", "direction",
    "ln_teu_volume", # Dependent variable
    "ln_freight_index", "ln_bunker", "ln_us_epu_index", # Main IVs
    "congestion_dummy", # Main IV (binary)
    "eu_ip_index", "uk_ip_index", "trade_imbalance", # Controls
    "ln_gbp_eur_rate", "ln_uk_cpi" # Controls
]
df = df.dropna(subset=cols_needed).copy()

# This Code is for modeling our per route

# 1. DEFINE FEATURES AND TARGET (This is kept the same for all of our routes)

# Our initial predictive Model
# ln(TEU volume) = freight prices + fuel costs + uncertainty + congestion + macro controls
#
# We have fit separate models for each route-direction combo to allow our coefficients aka elasticities
# be varying across all of our routes.

X_cols = [
    "ln_freight_index", # Freight rate
    "ln_bunker", # Fuel cost

```



```

    "ln_us_epu_index",      # Policy uncertainty
    "congestion_dummy",    # Congestion shock effect
    "eu_ip_index",         # EU demand
    "uk_ip_index",         # UK demand
    "trade_imbalance",     # Trade balance
    "ln_gbp_eur_rate",     # Exchange rate
    "ln_uk_cpi"            # Inflation
]
y_col = "ln_teu_volume"

routes = df["route"].unique()
directions = df["direction"].unique()

# This stores our results
route_results = [] # Per-route coefficients
all_preds = []     # Actual vs predicted values for all the different routes

# Train & test split by time: we train on data up until 2022, test on 2023-2024
train_end_year = 2022

```

In [95]: # 2. PER-ROUTE MODEL TRAINING AND EVALUATION

```

for route in routes:
    for direction in directions:
        # Initially we subset data for this specific route-direction
        sub = df[(df["route"] == route) & (df["direction"] == direction)].copy()
        sub = sub.sort_values("month")

        # We will skip it if we do not have at least 40 observations available, to ensure a large enough sample
        if len(sub) < 40:
            continue

        # Time-based train & test split
        train_mask = sub["month"].dt.year <= train_end_year
        test_mask = sub["month"].dt.year > train_end_year

        train = sub[train_mask] # Historical data (2016-2022)
        test = sub[test_mask]   # Recent data (2023-2024)

```

```
# Skip if train or test sets if we do not have a large enough data set
if len(train) < 30 or len(test) < 6:
    continue

# Extract the features
X_train = train[X_cols].values
y_train = train[y_col].values
X_test  = test[X_cols].values
y_test  = test[y_col].values

# FIT PER-ROUTE OLS MODEL

# Using sklearn we fit a separate model

model = LinearRegression()
model.fit(X_train, y_train)

# Then generating the predictions in log space
y_train_pred = model.predict(X_train) # In-sample predictions
y_test_pred  = model.predict(X_test)  # Out-of-sample predictions

# Using a Back-transform prediction from log to TEU levels
train_teu_actual = np.exp(y_train)
train_teu_pred   = np.exp(y_train_pred)
test_teu_actual  = np.exp(y_test)
test_teu_pred    = np.exp(y_test_pred)

# Estimating forecast accuracy

# Training metrics
train_rmse = np.sqrt(mean_squared_error(y_train, y_train_pred)) # RMSE in log space
train_r2    = r2_score(y_train, y_train_pred)                   # R2 in log space

# Test metrics
test_rmse   = np.sqrt(mean_squared_error(y_test, y_test_pred)) # RMSE in log space
```

```

test_r2      = r2_score(y_test, y_test_pred)                #  $R^2$  in log space

# MAPE (Mean Absolute Percentage Error)
# MAPE = average of (actual - predicted) / actual, as a percentage
test_mape    = np.mean(np.abs((test_teu_actual - test_teu_pred) / test_teu_actual)) * 100

# Printing the forecasted performance summary for this specific route-direction

print(f"ROUTE: {route} - {direction}")

print(f"Train N: {len(train)}, Test N: {len(test)}")
print(f"Train RMSE (ln TEU): {train_rmse:.4f},  $R^2$ : {train_r2:.3f}")
print(f"Test  RMSE (ln TEU): {test_rmse:.4f},  $R^2$ : {test_r2:.3f}")
print(f"Test  MAPE (TEU):    {test_mape:.2f}%")

# Storing the route-specific coefficients

# Extract coefficients for main explanatory variables
coef_series = pd.Series(model.coef_, index=X_cols)
coef_series["intercept"] = model.intercept_

# Storing coefficients for the four main IVs only
for var in ["ln_freight_index", "ln_bunker", "ln_us_epu_index", "congestion_dummy"]:
    route_results.append({
        "route": route,
        "direction": direction,
        "variable": var,
        "coef": coef_series[var] # Route-specific elasticity
    })

# Storing predictions for the plotting

# Save actual vs predicted values for visualization
temp_pred = test[["month", "route", "direction"]].copy()
temp_pred["ln_teu_actual"] = y_test          # Actual log TEU
temp_pred["ln_teu_pred"]   = y_test_pred     # Predicted log TEU

```

```
temp_pred["teu_actual"] = test_teu_actual # Actual TEU
temp_pred["teu_pred"] = test_teu_pred # Predicted TEU
all_preds.append(temp_pred)
```

```
ROUTE: UK-DE – Exports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.1140, R²: 0.440
Test RMSE (ln TEU): 0.2760, R²: -3.235
Test MAPE (TEU): 20.41%
ROUTE: UK-DE – Imports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.0897, R²: 0.691
Test RMSE (ln TEU): 0.1945, R²: -1.293
Test MAPE (TEU): 15.15%
ROUTE: UK-FR – Exports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.1130, R²: 0.636
Test RMSE (ln TEU): 0.3546, R²: -5.217
Test MAPE (TEU): 26.17%
ROUTE: UK-FR – Imports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.1030, R²: 0.756
Test RMSE (ln TEU): 0.2996, R²: -2.717
Test MAPE (TEU): 21.91%
ROUTE: UK-NL – Exports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.1016, R²: 0.763
Test RMSE (ln TEU): 0.1689, R²: -0.045
Test MAPE (TEU): 13.07%
ROUTE: UK-NL – Imports
Train N: 84, Test N: 33
Train RMSE (ln TEU): 0.0855, R²: 0.827
Test RMSE (ln TEU): 0.1636, R²: -0.250
Test MAPE (TEU): 12.01%
```

```
In [94]: # 3. Saving to a CSV

# Save route-specific coefficients (elasticities)
```

```

coef_df = pd.DataFrame(route_results)
if not coef_df.empty:
    coef_df.to_csv("PerRoute_OLS_Coefficients.csv", index=False)
    print("\nSaved: PerRoute_OLS_Coefficients.csv")
    print("\nRoute-specific elasticities (per-route coefficients):")
    # Pivot table: rows = route-direction, columns = variables, values = coefficients
    print(coef_df.pivot_table(index=["route", "direction"], columns="variable", values="coef").round(3))

# Save actual vs predicted values for all routes and test periods
if all_preds:
    preds_df = pd.concat(all_preds, ignore_index=True)
    preds_df.to_csv("PerRoute_OLS_TestPredictions.csv", index=False)
    print("\nSaved: PerRoute_OLS_TestPredictions.csv")

```

Saved: PerRoute_OLS_Coefficients.csv

Route-specific elasticities (per-route coefficients):

variable	congestion_dummy	ln_bunker	ln_freight_index \
route direction			
UK-DE Exports	-0.062	0.176	-0.050
Imports	-0.037	0.151	-0.035
UK-FR Exports	-0.099	0.354	-0.075
Imports	-0.110	0.313	-0.087
UK-NL Exports	-0.101	0.356	-0.081
Imports	-0.114	0.289	-0.071

variable	ln_us_epu_index
route direction	
UK-DE Exports	0.001
Imports	-0.010
UK-FR Exports	0.043
Imports	0.018
UK-NL Exports	0.020
Imports	0.025

Saved: PerRoute_OLS_TestPredictions.csv

In []: # Model 2 Predictive Model for 2024 Data Only

```

In [101... # This model is only predicting for 2024, and is done for focusing on the recent performance of the model.
# This is a further refinement on the previous model, just trying to map out some useable insights.

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# 0. Loading the data directly from our Github folder online

PANEL_URL = "https://raw.githubusercontent.com/truthisonegt-pixel/Data-ML-Repository-/main/Shipping_Volume_

df = pd.read_csv(PANEL_URL)
df["month"] = pd.to_datetime(df["month"])

# This code ensures we only keep the needed columns for regression and drop any rows with missing values
cols_needed = [
    "month", "route", "direction", "year",
    "ln_teu_volume", # Dependent variable
    "ln_freight_index", "ln_bunker", "ln_us_epu_index", # Main IVs
    "congestion_dummy", # Main IV (binary)
    "eu_ip_index", "uk_ip_index", "trade_imbalance", # Controls
    "ln_gbp_eur_rate", "ln_uk_cpi" # Controls
]
df = df.dropna(subset=cols_needed).copy()

```

```

In [100... # Prediction Model for 2024
# 1. FEATURES AND 2024 TRAIN/TEST SPLIT

# Train/test split:
# - Training the model based on the data between 2016-01 to 2023-12
# - Then testing for the time between 2024-01 to 2024-12

X_cols = [
    "ln_freight_index", # Freight rate elasticity
    "ln_bunker", # Fuel cost elasticity
    "ln_us_epu_index", # Policy uncertainty elasticity
    "congestion_dummy", # Congestion shock effect

```

```

    "eu_ip_index",      # EU demand
    "uk_ip_index",      # UK demand
    "trade_imbalance",  # Trade balance
    "ln_gbp_eur_rate",  # Exchange rate
    "ln_uk_cpi"         # Inflation
]
y_col = "ln_teu_volume" # Target: log of TEU volume

routes = df["route"].unique()
directions = df["direction"].unique()

forecast_rows = []

# Defining the train/test cutoff dates for our model
train_end = pd.Timestamp("2023-12-01") # Last month of training data
test_start = pd.Timestamp("2024-01-01") # First month of forecast period

```

In [98]: # 2. Per route 2024 forecast Generation

```

# For each route-direction:
# - First we train on all historical data present 2016-2023
# - Then we predict 2024 TEU volumes on a monthly basis
# - finally we calculated the forecast accuracy metrics for 2024 predictions

for route in routes:
    for direction in directions:
        # We substed the data for this specific route-direction
        sub = df[(df["route"] == route) & (df["direction"] == direction)].copy()
        sub = sub.sort_values("month")

        # We split into training and test sets
        train = sub[sub["month"] <= train_end]
        test = sub[sub["month"] >= test_start]

        # We skip if the training/test data is empty
        if train.empty or test.empty:
            continue

```

```
X_train = train[X_cols].values
y_train = train[y_col].values

X_test  = test[X_cols].values
y_test  = test[y_col].values

# FIT OLS MODEL ON 2016–2023 DATA

# This code trains the linear regression using all the available historical data
# with the objective of maximizing the accuracy of the code
model = LinearRegression()
model.fit(X_train, y_train)

# Generate 2024 predictions
y_pred_2024 = model.predict(X_test)

# Back-transform predictions and actuals from log to TEU levels
teu_actual_2024 = np.exp(y_test)      # Actual TEU volumes in 2024
teu_pred_2024   = np.exp(y_pred_2024) # Predicted TEU volumes in 2024

# CALCULATE 2024 FORECAST ACCURACY
# Test R^2
test_r2 = r2_score(y_test, y_pred_2024)
print(f"R² (ln TEU): {test_r2:.3f}")

# Metrics in the given Log space
rmse_ln = np.sqrt(mean_squared_error(y_test, y_pred_2024)) # Root Mean Squared Error (log)
mae_ln  = mean_absolute_error(y_test, y_pred_2024)         # Mean Absolute Error (log)

# Metrics in TEU levels (Makes it easier to interpret the results)
rmse_teu = np.sqrt(mean_squared_error(teu_actual_2024, teu_pred_2024)) # RMSE in TEUs
mae_teu  = mean_absolute_error(teu_actual_2024, teu_pred_2024)         # MAE in TEUs

# MAPE (Mean Absolute Percentage Error) – Forecast accuracy
# MAPE = average of (actual – predicted) / actual, expressed as percentage
mape_teu = np.mean(np.abs((teu_actual_2024 - teu_pred_2024) / teu_actual_2024)) * 100
```



```

from sklearn.metrics import r2_score # Add to imports

# After MAPE line:

# Printing the 2024 forecast performance summary for this route-direction

print(f"2024 FORECAST: {route} – {direction}")

print(f"N 2024 months: {len(test)}")
print(f"RMSE (ln TEU): {rmse_ln:.4f}")
print(f"MAE (ln TEU): {mae_ln:.4f}")
print(f"RMSE (TEU): {rmse_teu:,.0f}")
print(f"MAE (TEU): {mae_teu:,.0f}")
print(f"MAPE (TEU): {mape_teu:.2f}%")

# Storing the 2024 predictions for later graphing

# Save actual vs predicted values for each month in 2024
# This allows detailed visualization and analysis of forecast accuracy
temp = test[["month", "route", "direction"]].copy()
temp["ln_teu_actual"] = y_test # Actual log TEU
temp["ln_teu_pred"] = y_pred_2024 # Predicted log TEU
temp["teu_actual"] = teu_actual_2024 # Actual TEU
temp["teu_pred"] = teu_pred_2024 # Predicted TEU
forecast_rows.append(temp)

```

```

R² (ln TEU): -1.881
2024 FORECAST: UK-DE – Exports
N 2024 months: 21
RMSE (ln TEU): 0.2013
MAE (ln TEU): 0.1609
RMSE (TEU): 1,016
MAE (TEU): 791
MAPE (TEU): 14.27%
R² (ln TEU): -0.478
2024 FORECAST: UK-DE – Imports

```

N 2024 months: 21
RMSE (ln TEU): 0.1545
MAE (ln TEU): 0.1211
RMSE (TEU): 1,089
MAE (TEU): 854
MAPE (TEU): 11.01%
 R^2 (ln TEU): -2.734
2024 FORECAST: UK-FR – Exports
N 2024 months: 21
RMSE (ln TEU): 0.2710
MAE (ln TEU): 0.2390
RMSE (TEU): 978
MAE (TEU): 855
MAPE (TEU): 20.61%
 R^2 (ln TEU): -1.591
2024 FORECAST: UK-FR – Imports
N 2024 months: 21
RMSE (ln TEU): 0.2629
MAE (ln TEU): 0.2367
RMSE (TEU): 1,100
MAE (TEU): 987
MAPE (TEU): 20.56%
 R^2 (ln TEU): 0.198
2024 FORECAST: UK-NL – Exports
N 2024 months: 21
RMSE (ln TEU): 0.1677
MAE (ln TEU): 0.1451
RMSE (TEU): 759
MAE (TEU): 633
MAPE (TEU): 13.50%
 R^2 (ln TEU): -0.595
2024 FORECAST: UK-NL – Imports
N 2024 months: 21
RMSE (ln TEU): 0.1804
MAE (ln TEU): 0.1493
RMSE (TEU): 891
MAE (TEU): 733
MAPE (TEU): 13.53%

In [99]: *# 3. Saving the data as a CSV*

```
if forecast_rows:
    forecast_df = pd.concat(forecast_rows, ignore_index=True)
    forecast_df.to_csv("PerRoute_OLS_2024_Forecasts.csv", index=False)
    print("\nSaved: PerRoute_OLS_2024_Forecasts.csv")
else:
    print("No 2024 forecasts generated (check that 2024 exists in the data).")
```

Saved: PerRoute_OLS_2024_Forecasts.csv

In []: *# Visualizing the Predictions for 2024*

In [66]: **import** matplotlib.pyplot **as** plt

Loading all of our data from Github

```
FORECAST_URL = "https://raw.githubusercontent.com/truthisonet-pixel/Data-ML-Repository-/main/PerRoute_OLS_
```

```
fore = pd.read_csv(FORECAST_URL)
fore["month"] = pd.to_datetime(fore["month"])
```

Filter for 2024 only

```
fore_2024 = fore[fore["month"].dt.year == 2024].copy()
```

CREATE VISUALIZATION

*# This graph is aiming to highlight the accuracy of the predicted 2024 values of
the previous model against the actual data we collected*

```
fig, axes = plt.subplots(3, 2, figsize=(14, 10))
fig.suptitle('2024 TEU Volume: Actual vs Predicted by Route-Direction', fontsize=16, fontweight='bold')
```

We used the actual route names

```
routes = ["UK-DE", "UK-FR", "UK-NL"]
route_labels = ["UK-Germany", "UK-France", "UK-Netherlands"]
directions = ["Exports", "Imports"]
```

We are plotting the TEU against the time so 01-2024:12-2024

```

for i, (route, label) in enumerate(zip(routes, route_labels)):
    for j, direction in enumerate(directions):
        ax = axes[i, j]

        sub = fore_2024[(fore_2024["route"] == route) & (fore_2024["direction"] == direction)].copy()
        sub = sub.sort_values("month")

        if not sub.empty:
            # Plot actual and predicted
            ax.plot(sub["month"], sub["teu_actual"], marker='o', label='Actual',
                    linewidth=2, color='#2E86AB', markersize=6)
            ax.plot(sub["month"], sub["teu_pred"], marker='s', label='Predicted',
                    linewidth=2, linestyle='--', color='#A23B72', markersize=6)

            # Calculate MAPE
            mape = np.mean(np.abs((sub["teu_actual"] - sub["teu_pred"]) / sub["teu_actual"])) * 100

            ax.set_title(f"{label} - {direction}\nMAPE: {mape:.1f}%",
                        fontsize=11, fontweight='bold')
            ax.set_xlabel("Month", fontsize=9)
            ax.set_ylabel("TEU Volume", fontsize=9)
            ax.legend(loc='best', fontsize=8)
            ax.grid(alpha=0.3)
            ax.tick_params(axis='x', rotation=45, labelsiz=8)

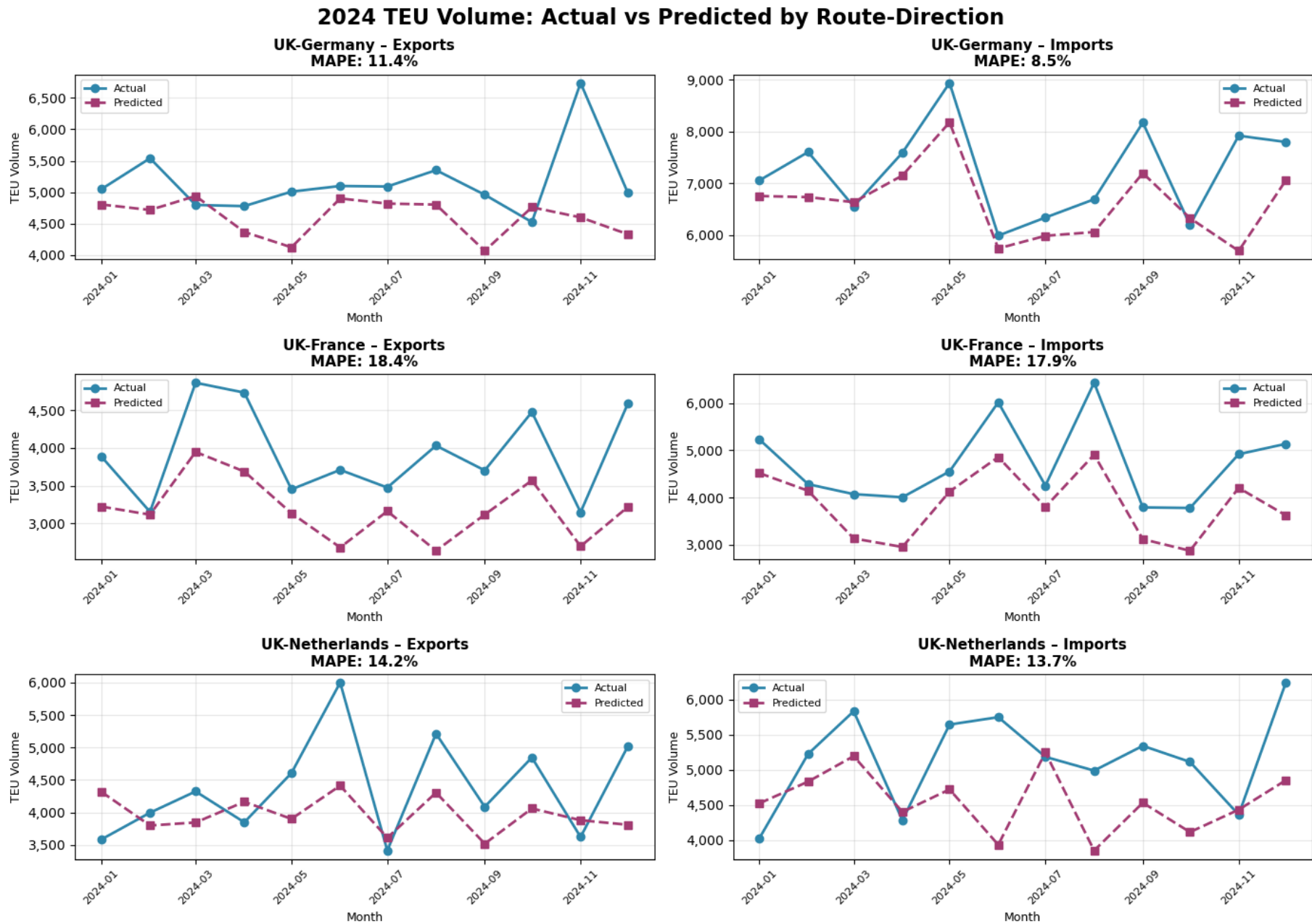
            # Format y-axis with commas for thousands
            ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'{int(x):,}'))
        else:
            ax.text(0.5, 0.5, 'No data', ha='center', va='center',
                    transform=ax.transAxes, fontsize=12)
            ax.set_title(f"{label} - {direction}", fontsize=11)
            ax.set_xlim(0, 1)
            ax.set_ylim(0, 1)

plt.tight_layout()
plt.savefig('2024_Forecasts_ActualVsPredicted.png', dpi=300, bbox_inches='tight')
print("\n✓ Saved: 2024_Forecasts_ActualVsPredicted.png")
plt.show()

```

2024 data only: 72 rows

✓ Saved: 2024_Forecasts_ActualVsPredicted.png



In []: *#Visualizing the Correlations Between Variables in the Dataset*

```
In [77]: # Load our panel data from the github to create the correlation matrix
PANEL_URL = "https://raw.githubusercontent.com/truthisonegt-pixel/Data-ML-Repository-/main/Shipping_Volume_
df = pd.read_csv(PANEL_URL)

# Select key variables for correlation. These are the main variables in our model
vars_to_corr = [
    "ln_teu_volume",
    "ln_freight_index",
    "ln_bunker",
    "ln_us_epu_index",
    "congestion_dummy",
    "eu_ip_index",
    "uk_ip_index",
    "trade_imbalance",
    "ln_gbp_eur_rate",
    "ln_uk_cpi"
]

# Compute correlation matrix
corr_matrix = df[vars_to_corr].corr()

# Plot heatmap
fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            square=True, linewidths=1, cbar_kws={"shrink": 0.8},
            vmin=-1, vmax=1, ax=ax)

ax.set_title('Correlation Matrix of Key Variables', fontsize=14, fontweight='bold', pad=20)

# Clean up labels
labels = [
    "ln(TEU Volume)",
    "ln(Freight Index)",
    "ln(Bunker)",
    "ln(US EPU)",
    "Congestion",
    "EU IP",
```

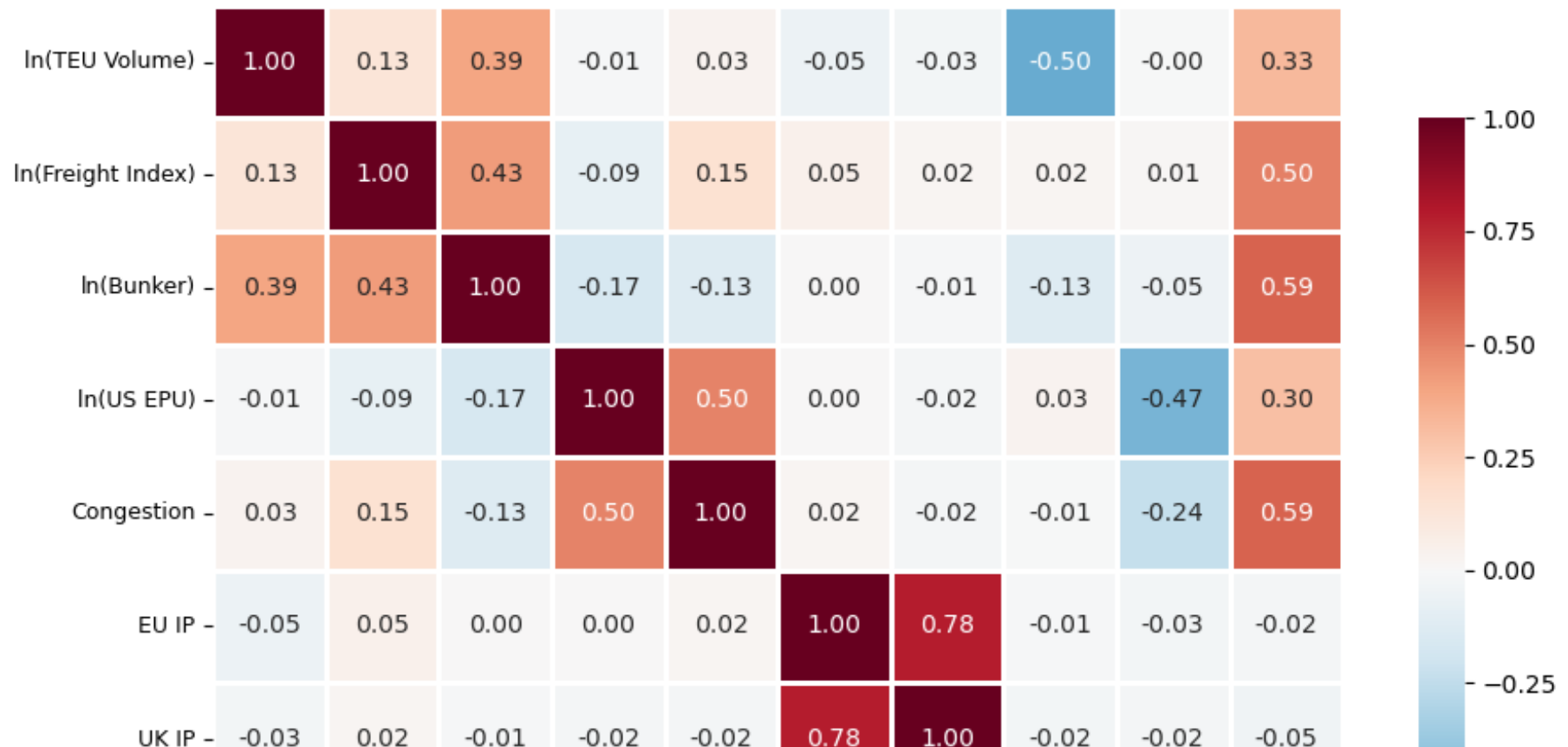
```

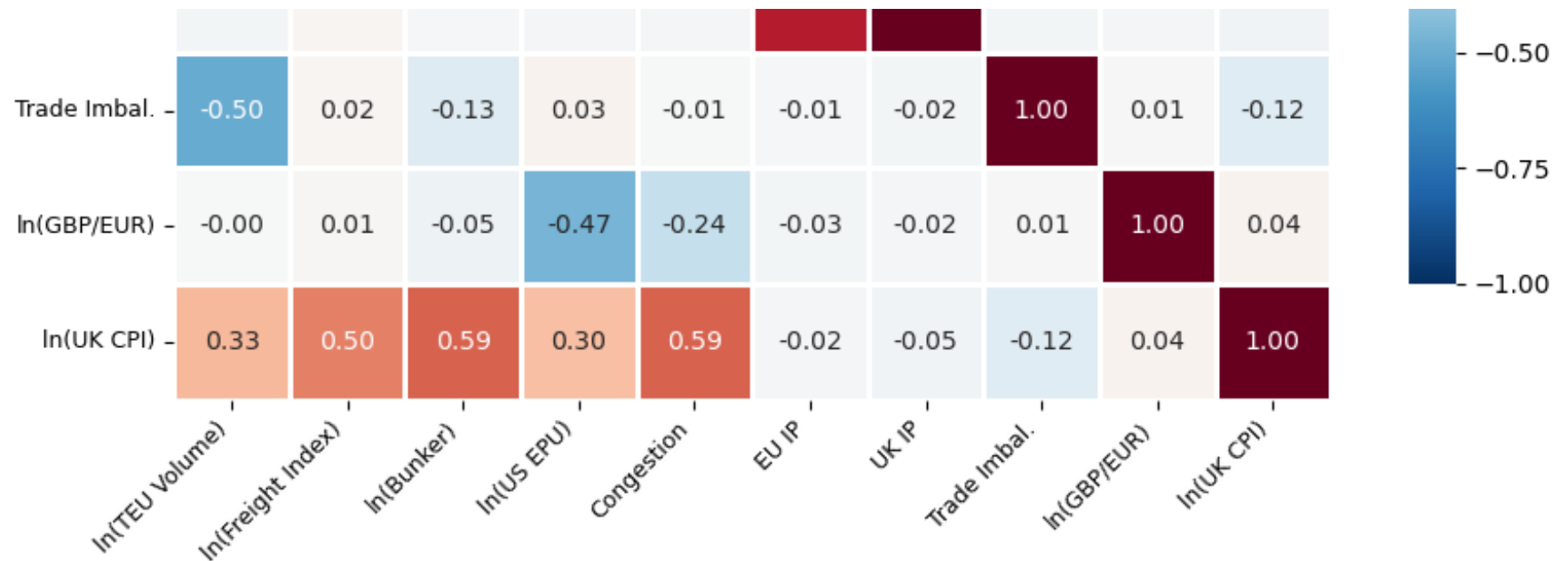
    "UK IP",
    "Trade Imbal.",
    "ln(GBP/EUR)",
    "ln(UK CPI)"
]
ax.set_xticklabels(labels, rotation=45, ha='right', fontsize=9)
ax.set_yticklabels(labels, rotation=0, fontsize=9)

plt.tight_layout()
plt.savefig('Correlation_Matrix.png', dpi=300, bbox_inches='tight')
plt.show()

```

Correlation Matrix of Key Variables





```
In [ ]: # Visualizing the Elasticities of the Variables in the Dataset
```

```
In [68]: # Load the data directly from our Github folder online
```

```
COEF_URL = "https://raw.githubusercontent.com/truthisonet-pixel/Data-ML-Repository-/main/PerRoute_OLS_Coefficients.csv"

# Load coefficients
coef_df = pd.read_csv(COEF_URL)

# Creating the actual Visuals

# Variables
variables = ["ln_freight_index", "ln_bunker", "ln_us_epu_index", "congestion_dummy"]
var_labels = ["Freight Index", "Bunker Price", "US EPU Index", "Congestion"]

# Create route Level Labels for Display
route_map = {
    "UK-DE": "UK-Germany",
    "UK-FR": "UK-France",
```

```

    "UK-NL": "UK-Netherlands"
}

# Create 2x2 subplots
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Route-Specific Elasticities by Variable (Exports)', fontsize=16, fontweight='bold')

for idx, (var, label) in enumerate(zip(variables, var_labels)):
    ax = axes[idx // 2, idx % 2]

    # Filter for this variable and Exports only
    sub = coef_df[(coef_df["variable"] == var) & (coef_df["direction"] == "Exports")].copy()

    if not sub.empty:
        # Sort by coefficient value
        sub = sub.sort_values("coef")

        # Map route codes to full names for display
        sub["route_label"] = sub["route"].map(route_map)

        # Color code: positive = blue, negative = red
        colors = ['#2E86AB' if x > 0 else '#D62828' for x in sub["coef"]]

        ax.barh(sub["route_label"], sub["coef"], color=colors, edgecolor='black', linewidth=0.8)
        ax.axvline(0, color='black', linewidth=1.2)
        ax.set_xlabel("Elasticity (Coefficient)", fontsize=10)
        ax.set_title(f"{label} - Exports", fontsize=12, fontweight='bold')
        ax.grid(axis='x', alpha=0.3)

        # Add value labels on bars
        for i, (route, coef) in enumerate(zip(sub["route_label"], sub["coef"])):
            # Position text slightly inside or outside the bar
            offset = 0.005 if coef > 0 else -0.005
            ha = 'left' if coef > 0 else 'right'
            ax.text(coef + offset, i, f'{coef:.3f}', va='center', ha=ha,
                    fontsize=9, fontweight='bold')
    else:
        ax.text(0.5, 0.5, 'No data', ha='center', va='center', transform=ax.transAxes)

```

```

    ax.set_title(label, fontsize=12)

plt.tight_layout()
plt.savefig('Route_Specific_Elasticities_Exports.png', dpi=300, bbox_inches='tight')
print("\n✓ Saved: Route_Specific_Elasticities_Exports.png")
plt.show()

# CREATE IMPORTS VERSION

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
fig.suptitle('Route-Specific Elasticities by Variable (Imports)', fontsize=16, fontweight='bold')

for idx, (var, label) in enumerate(zip(variables, var_labels)):
    ax = axes[idx // 2, idx % 2]

    # Filter for this variable and Imports
    sub = coef_df[(coef_df["variable"] == var) & (coef_df["direction"] == "Imports")].copy()

    if not sub.empty:
        # Sort by coefficient value
        sub = sub.sort_values("coef")

        # Map route codes to full names for display
        sub["route_label"] = sub["route"].map(route_map)

        # Color code: positive = blue, negative = red
        colors = ['#2E86AB' if x > 0 else '#D62828' for x in sub["coef"]]

        ax.barh(sub["route_label"], sub["coef"], color=colors, edgecolor='black', linewidth=0.8)
        ax.axvline(0, color='black', linewidth=1.2)
        ax.set_xlabel("Elasticity (Coefficient)", fontsize=10)
        ax.set_title(f"{label} - Imports", fontsize=12, fontweight='bold')
        ax.grid(axis='x', alpha=0.3)

        # Add value labels on bars
        for i, (route, coef) in enumerate(zip(sub["route_label"], sub["coef"])):
            offset = 0.005 if coef > 0 else -0.005
            ha = 'left' if coef > 0 else 'right'

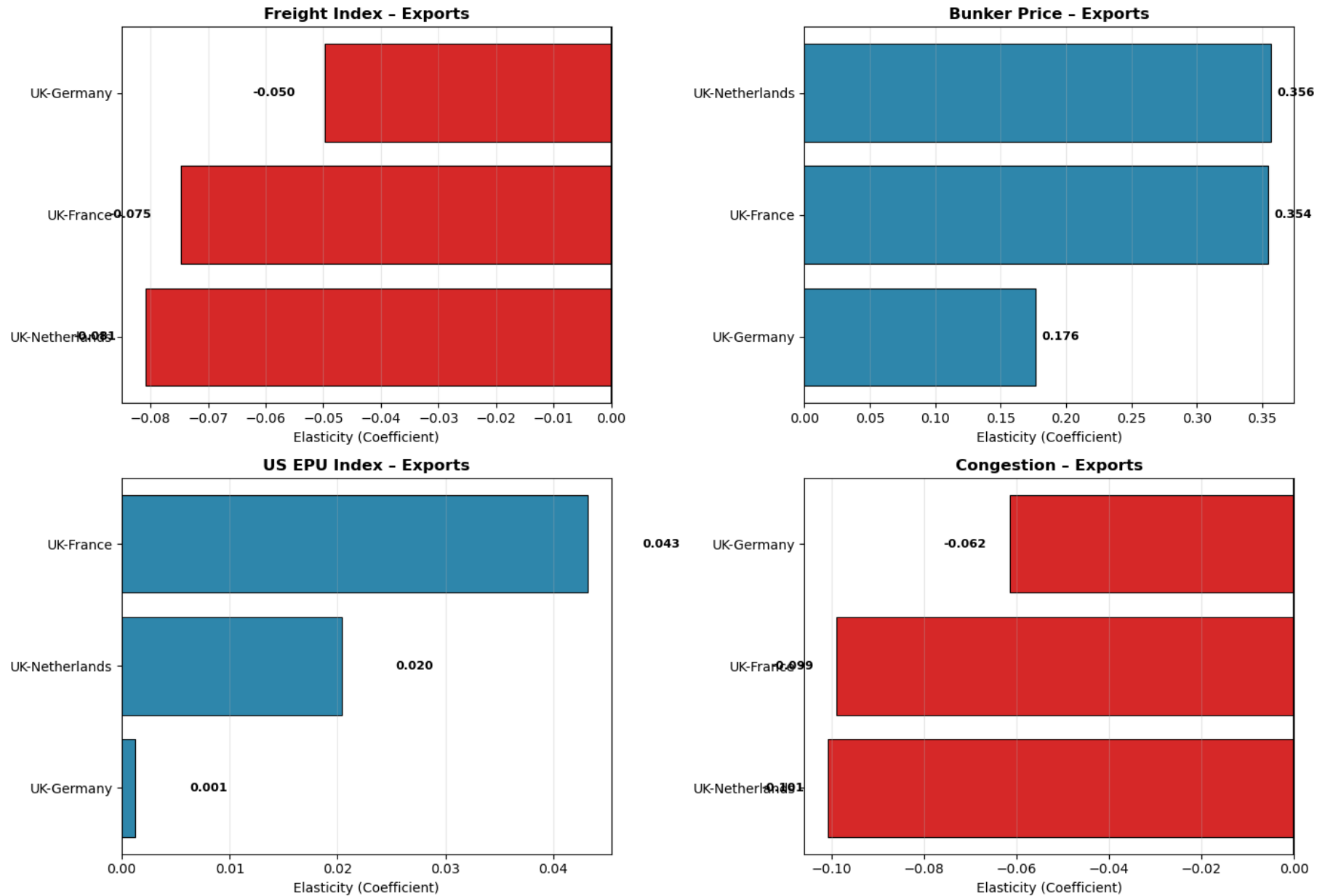
```

```
        ax.text(coef + offset, i, f'{coef:.3f}', va='center', ha=ha,
                fontsize=9, fontweight='bold')
    else:
        ax.text(0.5, 0.5, 'No data', ha='center', va='center', transform=ax.transAxes)
        ax.set_title(label, fontsize=12)

plt.tight_layout()
plt.savefig('Route_Specific_Elasticities_Imports.png', dpi=300, bbox_inches='tight')
print("✓ Saved: Route_Specific_Elasticities_Imports.png\n")
plt.show()
```

✓ Saved: Route_Specific_Elasticities_Exports.png

Route-Specific Elasticities by Variable (Exports)



✓ Saved: Route_Specific_Elasticities_Imports.png

Route-Specific Elasticities by Variable (Imports)

